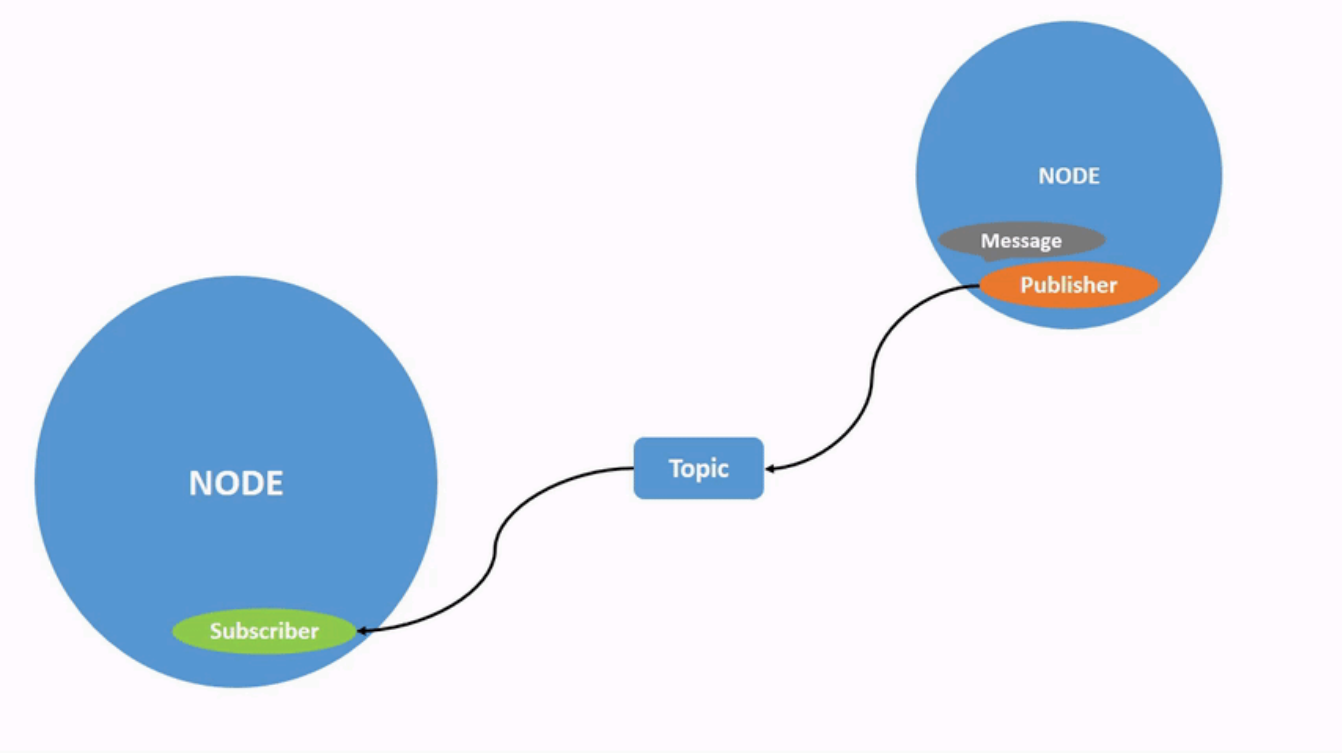
7. ROS2 Topic Communication

1. Introduction to Topic Communication

Topic communication is the most frequently used communication method in ROS2. A publisher publishes the data of a specified topic, and a subscriber only needs to subscribe to that topic's data to receive it.

Topic communication is based on the publish/subscribe model, as shown in the figure:



The characteristic of topic data transmission is that it goes from one node to another node. The object that sends the data is called the **publisher**, and the object that receives the data is called the **subscriber**. Every topic needs a name, and the transmitted data also needs a fixed data type.

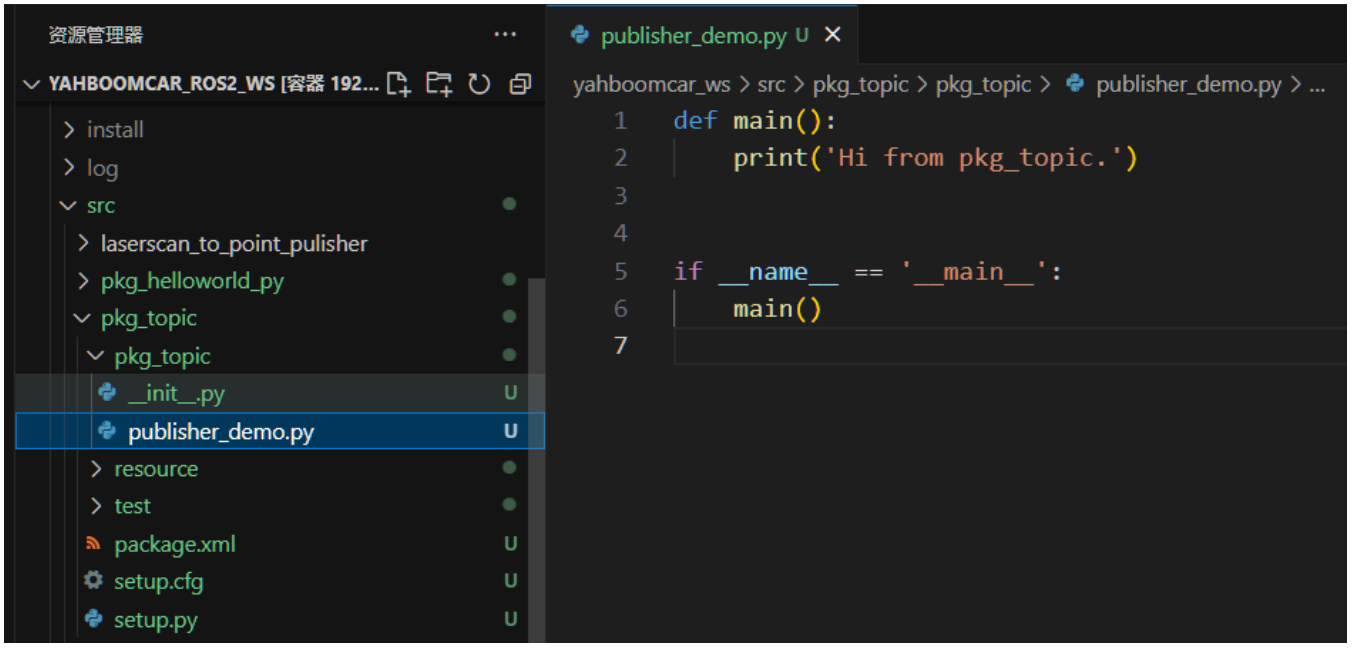
Next, how to implement the topic communication between nodes with the Python language will be explained.

2. Creating a New Function Package

- Switch to the src directory of the workspace
- Create the pkg_topic function package

```
ros2 pkg create pkg_topic --build-type ament_python --dependencies rclpy --node-name
publisher_demo
```

After executing the command above, the pkg_topic function package is created, a publisher_demo node is created at the same time, and the relevant configuration files have already been set up.



3. Publisher Implementation

3.1 Create the Publisher

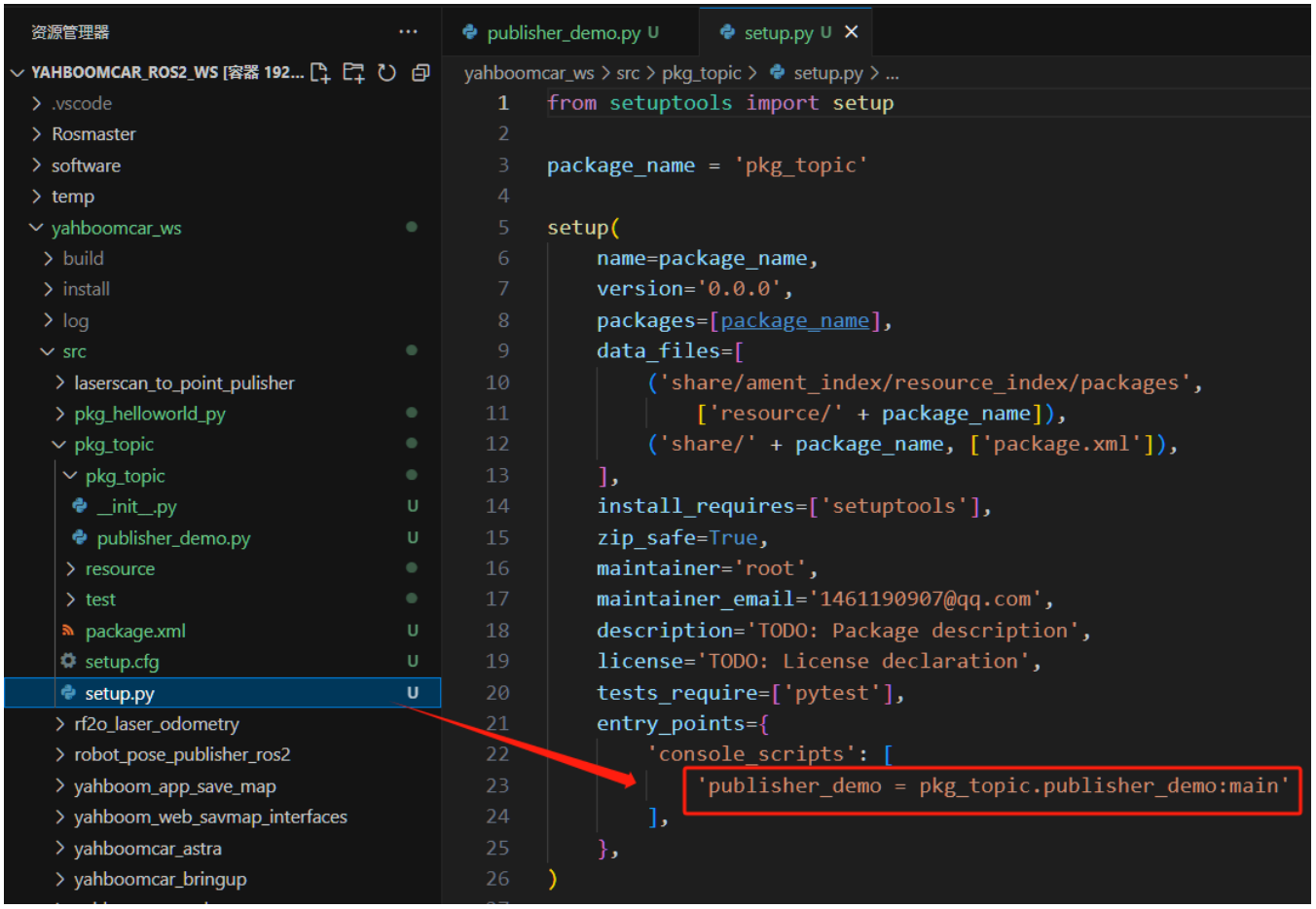
Next, edit [publisher_demo.py] to implement the publisher function and add the following code:

```
#Import the rclpy library
import rclpy
from rclpy.node import Node
#Import the String message
from std_msgs.msg import String
#Create a Topic_Pub node subclass that inherits from the Node base class, passing in one
parameter name
class Topic_Pub(Node):
    def __init__(self,name):
        super().__init__(name)
        # Create a publisher using the create_publisher function; the parameters passed in
are respectively:
        # the topic data type, the topic name, the queue length for storing the messages
        self.pub = self.create_publisher(String,"/topic_demo",1)
        # Create a timer with a 1s interval that enters the callback function; the
parameters passed in are respectively:
        # the execution interval of the callback function, the callback function
        self.timer = self.create_timer(1,self.pub_msg)
    # Define the callback function
    def pub_msg(self):
        msg = String()  # Create a variable msg of the String type
        msg.data = "Hi,I send a message." # Assign the data inside msg
        self.pub.publish(msg) # Publish the topic data

#Main function
def main():
    rclpy.init() # Initialize
    pub_demo = Topic_Pub("publisher_node") # Create an object of the Topic_Pub class; the
parameter passed in is the name of the node
```

```
    rclpy.spin(pub_demo)      # Run the rclpy.spin function, passing in one parameter, which
is the Topic_Pub object just created
    pub_demo.destroy_node()   # Destroy the node object
    rclpy.shutdown()          # Shut down the ROS2 Python interface
```

3.2 Edit the Configuration File

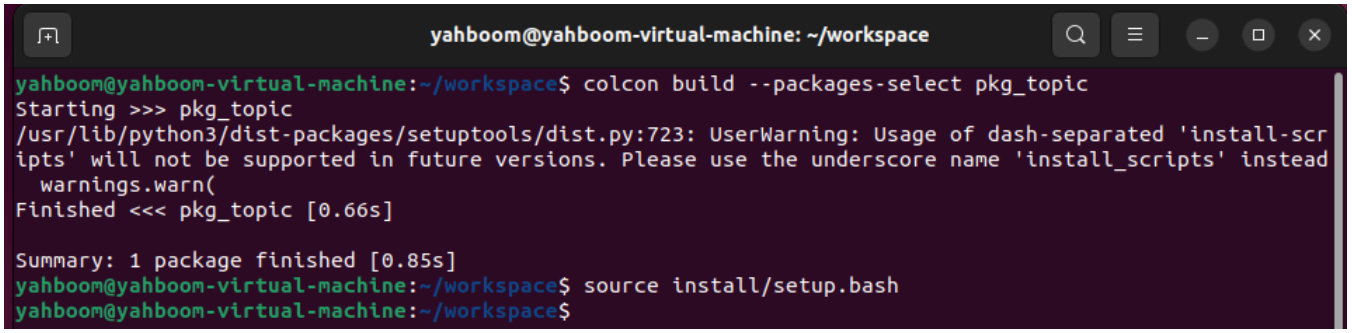


3.3 Build the Function Package

- Build the function package

```
colcon build --packages-select pkg_topic
```

- Refresh the environment variables under the workspace



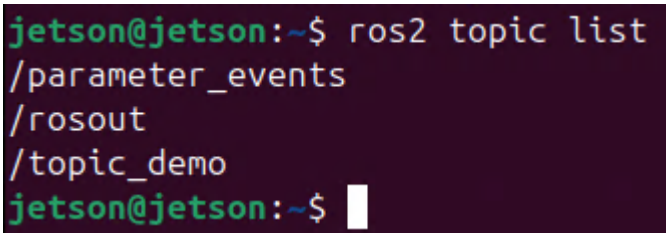
3.4 Run the Program

- Run the command after refreshing the environment variables

```
ros2 run pkg_topic publisher_demo
```

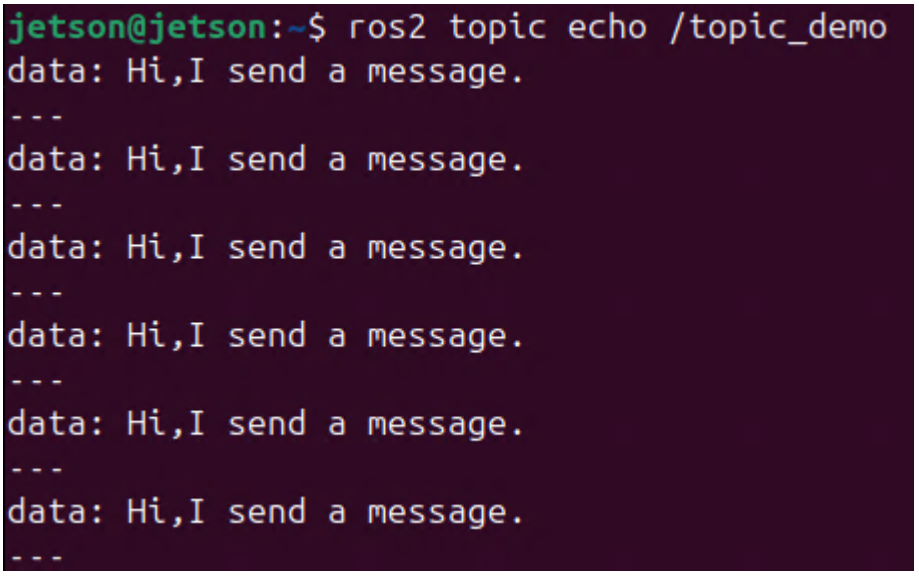
Nothing is printed after the program runs successfully, so we can use the ros2 topic tool to look at the data. First, check whether this topic is being published, open another terminal and enter:

```
ros2 topic list
```



This topic_demo is the topic data defined in the program. Next, let's use ros2 topic echo to print this data, enter in the terminal:

```
ros2 topic echo /topic_demo
```

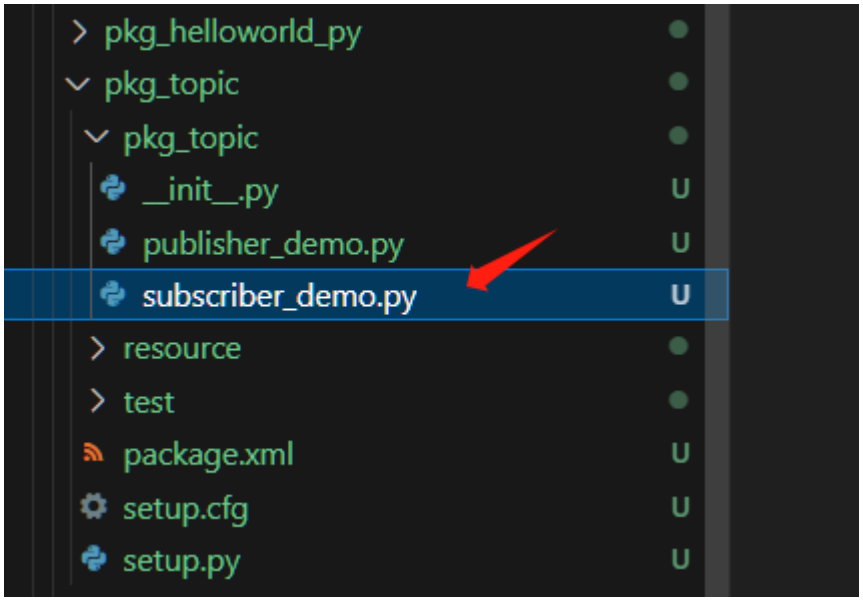


As can be seen, the "Hi,I send a message." printed in the terminal is consistent with the msg.data = "Hi,I send a message." in our code.

4. Subscriber Implementation

4.1 Create the Subscriber

Create a new file [subscriber_demo.py] in the same directory as [publisher_demo.py]



Next, edit [subscriber_demo.py] to implement the subscriber function and add the following code:

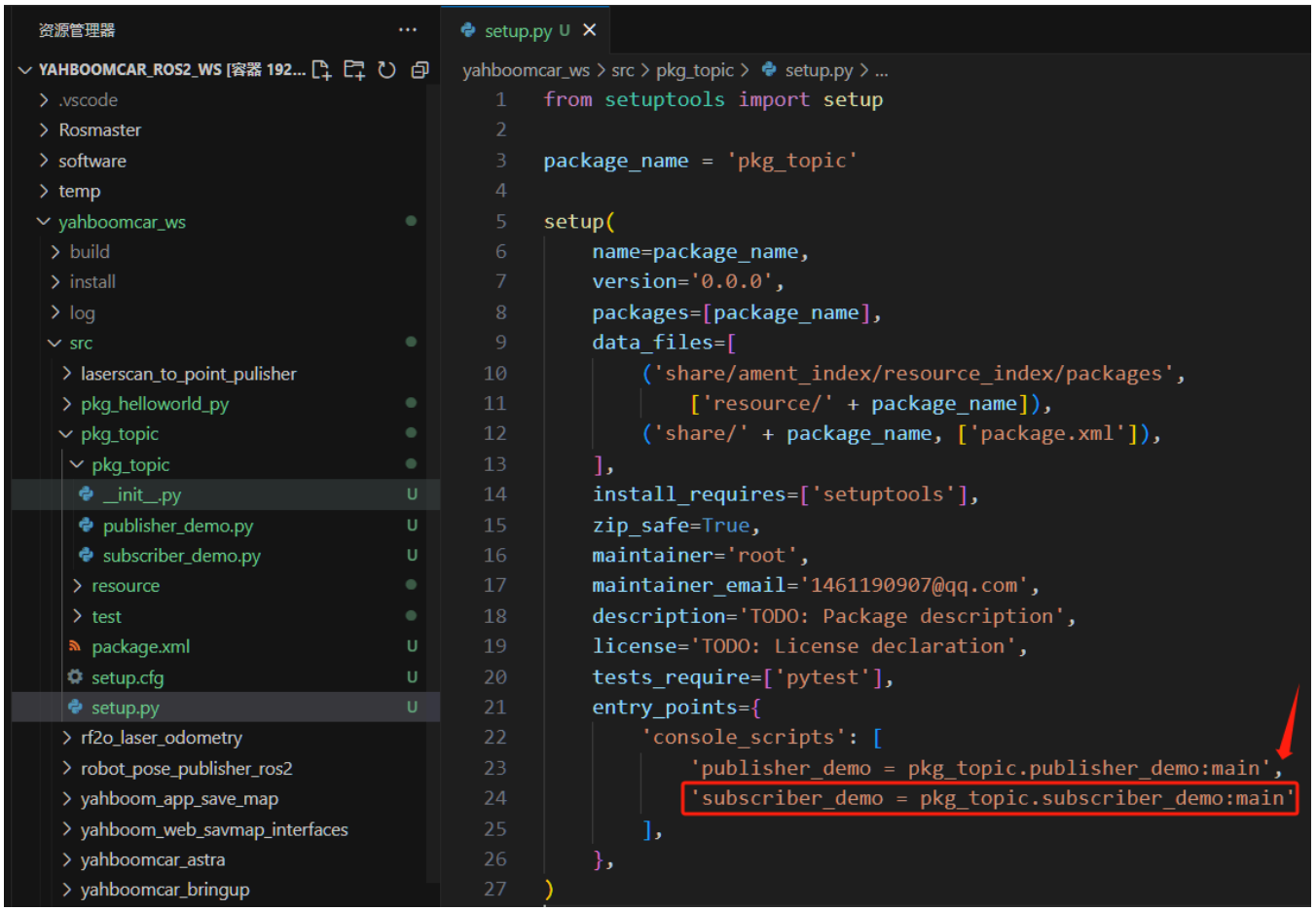
```
#Import the relevant libraries
import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class Topic_Sub(Node):
    def __init__(self,name):
        super().__init__(name)
        # Creating a subscriber uses create_subscription; the parameters passed in are
respectively: the topic data type, the topic name, the callback function name, the queue
length
        self.sub = self.create_subscription(String,"/topic_demo",self.sub_callback,1)
    # The callback function executes the program: print the received information
    def sub_callback(self,msg):
        # print(msg.data,flush=True)
        self.get_logger().info(msg.data)



def main():
    rclpy.init() # ROS2 Python interface initialization
    sub_demo = Topic_Sub("subscriber_node") # Create the object and initialize it
    rclpy.spin(sub_demo)
    sub_demo.destroy_node()  # Destroy the node object
    rclpy.shutdown()         # Shut down the ROS2 Python interface
```

4.2 Edit the Configuration File

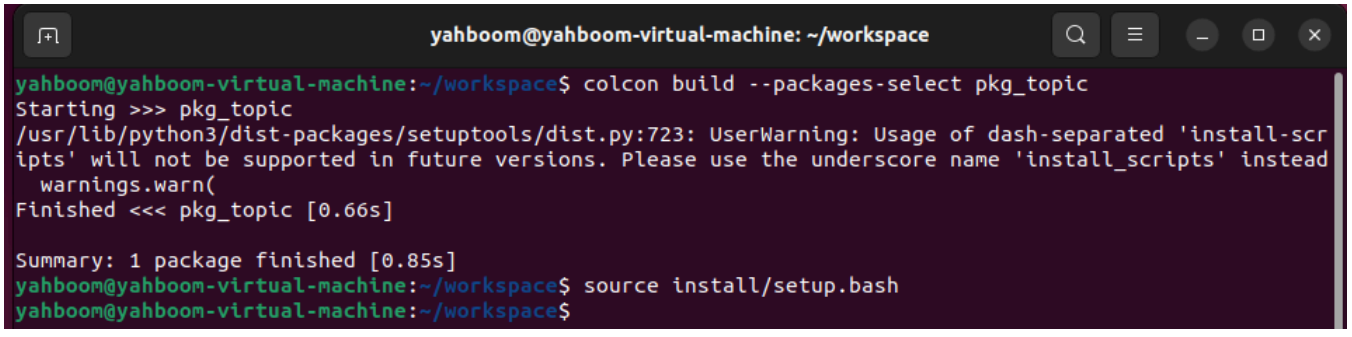


4.3 Build the Workspace

- Build the function package

```
colcon build --packages-select pkg_topic
```

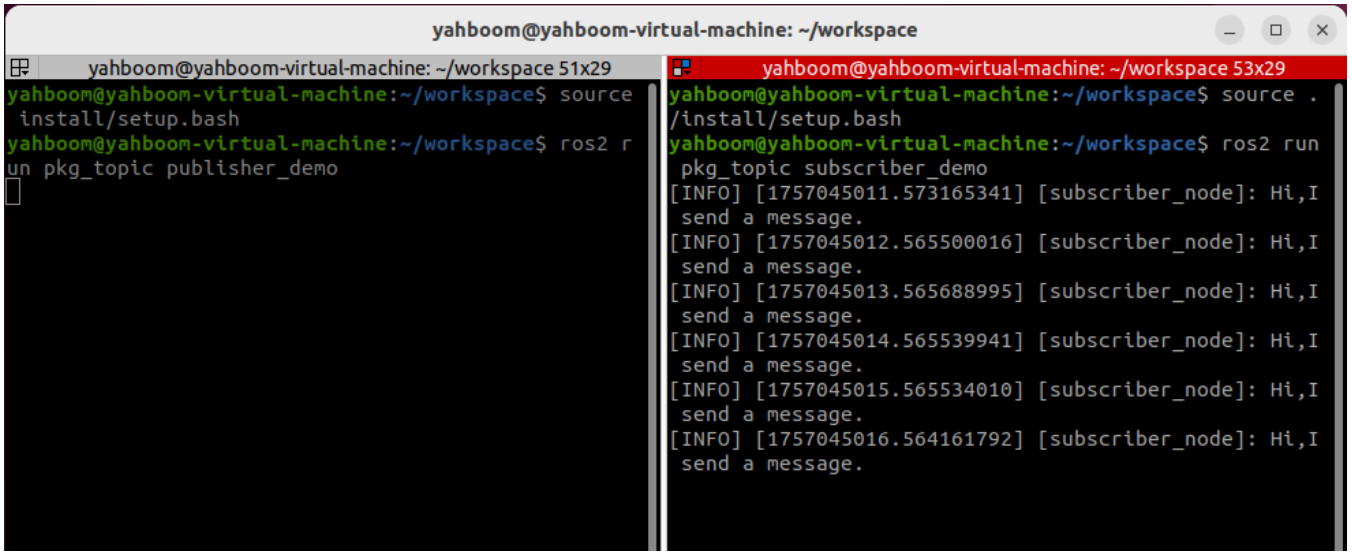
- Refresh the environment variables under the workspace



4.4 Run the Program

Split the terminal and execute the following:

```
#Start the publisher node
ros2 run pkg_topic publisher_demo
#Start the subscriber node
ros2 run pkg_topic subscriber_demo
```



As shown in the figure above, the terminal running the subscriber prints the information of /topic_demo published by the publisher.